



**PONTIFICIA UNIVERSIDAD JAVERIANA – CALI**  
**FACULTAD DE INGENIERÍA Y CIENCIAS**  
 ASIGNATURA: Principios de Sistemas Computacionales

PROFESOR: Mauricio Cano

**ACTIVIDAD DE RECUPERACIÓN #3**

HECHO POR: Nicolás Carrillo R. | Código 8991633

**1. Sobre el proyecto**

Se implementa un simulador de máquina de Turing de una sola cinta que carga una TM, la ejecuta paso a paso y devuelve ACCEPT, REJECT o TIMEOUT según se llegue al estado de aceptación, rechazo o se supere un límite de pasos. Además, se incluyeron tres TMs: un decider para  $0^n1^n$ , un decider para  $\{w#w\}$  y una TM que calcula el sucesor unario.

**2. TM para  $0^n1^n$**

La máquina sigue la estrategia que se vio anteriormente en clase: marca el 0 más a la izquierda con X, busca el primer 1 no marcado para marcarlo también con X y luego regresa al inicio para repetir. Si termina sin símbolos sueltos, acepta; si encuentra un desbalance o un orden incorrecto, rechaza.

**3. Trace sobre 000111 (accept)**

Resultado final: ACCEPT

| Paso | Configuración | Regla          |
|------|---------------|----------------|
| 0    | q1000111      | start          |
| 1    | Xq200111      | q1 0 -> q2 X R |
| 2    | X0q20111      | q2 0 -> q2 0 R |
| 3    | X00q2111      | q2 0 -> q2 0 R |
| 4    | X0q30X11      | q2 1 -> q3 X L |
| 5    | Xq300X11      | q3 0 -> q3 0 L |
| 6    | q3X00X11      | q3 0 -> q3 0 L |
| 7    | q3_X00X11     | q3 X -> q3 X L |
| 8    | _q1X00X11     | q3 _ -> q1 _ R |
| 9    | _Xq100X11     | q1 X -> q1 X R |
| 10   | _XXq20X11     | q1 0 -> q2 X R |
| 11   | _XX0q2X11     | q2 0 -> q2 0 R |
| 12   | _XX0Xq211     | q2 X -> q2 X R |
| 13   | _XX0q3XX1     | q2 1 -> q3 X L |
| 14   | _XXq30XX1     | q3 X -> q3 X L |
| 15   | _Xq3X0XX1     | q3 0 -> q3 0 L |
| 16   | _q3XX0XX1     | q3 X -> q3 X L |
| 17   | q3_XX0XX1     | q3 X -> q3 X L |
| 18   | _q1XX0XX1     | q3 _ -> q1 _ R |
| 19   | _Xq1X0XX1     | q1 X -> q1 X R |
| 20   | _XXq10XX1     | q1 X -> q1 X R |
| 21   | _XXXq2XX1     | q1 0 -> q2 X R |
| 22   | _XXXXq2X1     | q2 X -> q2 X R |
| 23   | _XXXXXq21     | q2 X -> q2 X R |
| 24   | _XXXXXq3XX    | q2 1 -> q3 X L |

|    |                |                 |
|----|----------------|-----------------|
| 25 | XXXq3XXX       | q3 X -> q3 X L  |
| 26 | XXq3XXXX       | q3 X -> q3 X L  |
| 27 | Xq3XXXXX       | q3 X -> q3 X L  |
| 28 | q3XXXXXX       | q3 X -> q3 X L  |
| 29 | q3 XXXXXX      | q3 X -> q3 X L  |
| 30 | q1XXXXXX       | q3 -> q1 R      |
| 31 | Xq1XXXXX       | q1 X -> q1 X R  |
| 32 | XXq1XXXX       | q1 X -> q1 X R  |
| 33 | XXXq1XXX       | q1 X -> q1 X R  |
| 34 | XXXXq1XX       | q1 X -> q1 X R  |
| 35 | XXXXXq1X       | q1 X -> q1 X R  |
| 36 | XXXXXXq1       | q1 X -> q1 X R  |
| 37 | XXXXXX qaccept | q1 -> qaccept R |

#### 4. Caso 0011

Para la cadena (equivoca en el documento) 0011 rotulado como reject. La cual se contradice por la definición de  $0^n 1^n$  como la traza mostrada en clase, donde 0011 sí acepta. Por esa razón, el simulador entrega el siguiente resultado:

Resultado final para 0011: ACCEPT

| Paso | Configuración | Regla           |
|------|---------------|-----------------|
| 0    | q10011        | start           |
| 1    | Xq2011        | q1 0 -> q2 X R  |
| 2    | X0q211        | q2 0 -> q2 0 R  |
| 3    | Xq30X1        | q2 1 -> q3 X L  |
| 4    | q3X0X1        | q3 0 -> q3 0 L  |
| 5    | q3 X0X1       | q3 X -> q3 X L  |
| 6    | q1X0X1        | q3 -> q1 R      |
| 7    | Xq10X1        | q1 X -> q1 X R  |
| 8    | XXq2X1        | q1 0 -> q2 X R  |
| 9    | XXXq21        | q2 X -> q2 X R  |
| 10   | XXq3XX        | q2 1 -> q3 X L  |
| 11   | Xq3XXX        | q3 X -> q3 X L  |
| 12   | q3XXXX        | q3 X -> q3 X L  |
| 13   | q3 XXXX       | q3 X -> q3 X L  |
| 14   | q1XXXX        | q3 -> q1 R      |
| 15   | Xq1XXX        | q1 X -> q1 X R  |
| 16   | XXq1XX        | q1 X -> q1 X R  |
| 17   | XXXq1X        | q1 X -> q1 X R  |
| 18   | XXXXq1        | q1 X -> q1 X R  |
| 19   | XXXX qaccept  | q1 -> qaccept R |

Como ejemplo de rechazo real para esta misma TM, la cadena 0110 produce REJECT, porque no tiene la forma  $0^n 1^n$ .

#### 5. Sobre indecidibilidad y max\_steps

Aunque el simulador realizado puede cargar una máquina de Turing y ejecutarla paso por paso, eso no significa que pueda saber siempre desde el principio, si esa máquina se va a detener o no para cualquier entrada. En casos sencillos sí se puede ver, porque la máquina llega a un estado de aceptación o de rechazo después de cierto número de pasos. Pero en general no existe un método formal de mirar cualquier par  $\langle M, w \rangle$  y garantizar si esa máquina va a terminar o se va a quedar ejecutándose para siempre.

Eso se conecta directamente con el halting problem, que justamente dice que no existe un algoritmo general que resuelva ese problema para todos los casos posibles. Es decir, no hay un procedimiento que funcione siempre y que pueda responder correctamente “sí se detiene” o “no se detiene” para cualquier máquina de Turing y cualquier entrada. Algunas máquinas aceptan, otras rechazan, pero otras simplemente entran en un ciclo infinito y nunca dan una respuesta final. Ahí está la diferencia entre poder simular una máquina concreta y poder decidir, en general, el comportamiento de todas las máquinas posibles.

Por eso se implementa max\_steps el cual fue necesario en el proyecto. Ese límite sirve para que el simulador no quede atrapado ejecutando una máquina indefinidamente. Si la máquina alcanza aceptación o rechazo antes de ese límite, el simulador responde normalmente. Pero si se llega al máximo de pasos sin que la máquina se detenga, el resultado es TIMEOUT. Ese resultado no

quiere decir que la máquina rechace, ni tampoco que necesariamente vaya a seguir para siempre, solo quiere decir que dentro del límite que nosotros dimos, no se detuvo.

Realmente `max_steps` no soluciona el problema de fondo. Más bien es una forma eficaz de combatir el hecho de que no podemos decidir en general si toda máquina se detiene o no, ponemos un límite finito para poder experimentar, probar y observar comportamientos sin que el programa se quede corriendo eternamente. Es por eso que con el simulador se ve muy bien una idea importante que deja el fragmento final del curso: hay problemas que sí se pueden ejecutar y observar en casos concretos, pero eso no significa que exista un método general para resolverlos siempre. Esa es precisamente una de las fronteras más importantes de la computación.